Vector Database

為什麼需要 Vector Database？

這幾年只要有在關注技術發展，幾乎都會看到「embeddings（向量嵌入）」這個詞。不管是能理解語意的搜尋引擎、精準得可怕的推薦系統、還是可以從大量文件中找資料的聊天機器人，本質上都在做一件事：快速找出「相似的東西」。

其實這個概念不新。推薦系統早就大量使用向量與相似度計算。但隨著新的機器學習模型（尤其是大語言模型與多模態模型）的成熟，向量資料庫的重要性被放大，並開始出現在各種基礎架構設計裡。

傳統資料庫很擅長做「精準查詢」e.g.

- 給我 user_id = 12345 的使用者
- 找出 1 月 1 日下的所有訂單

但如果你問：「找出和這篇文章相似的文件？」傳統資料庫就不擅長了。這正是向量資料庫存在的意義。

本講義會說明：

1. 向量是什麼
2. 相似度怎麼計算
3. KNN 與 ANN 的差異
4. 常見索引演算法（HNSW、IVF、LSH、Annoy）
5. 過濾（filter）與混合搜尋（hybrid search）
6. 實務中的更新與重建策略
7. 面試中怎麼談向量資料庫

如果你覺得索引演算法太細，可以先跳到應用場景再回來看。大多數系統設計面試並不要求你能實作 HNSW，而是希望你知道什麼時候該用向量資料庫。

什麼是 Vector（Embedding）？

Vector（向量）本質上就是一組數字陣列，用來表示某個「東西」。

這個東西可以是：

- 單字

buildmoat.org

- 句子
- 圖片
- 使用者
- 商品
- 任何可以丟進 ML 模型的資料

e.g.

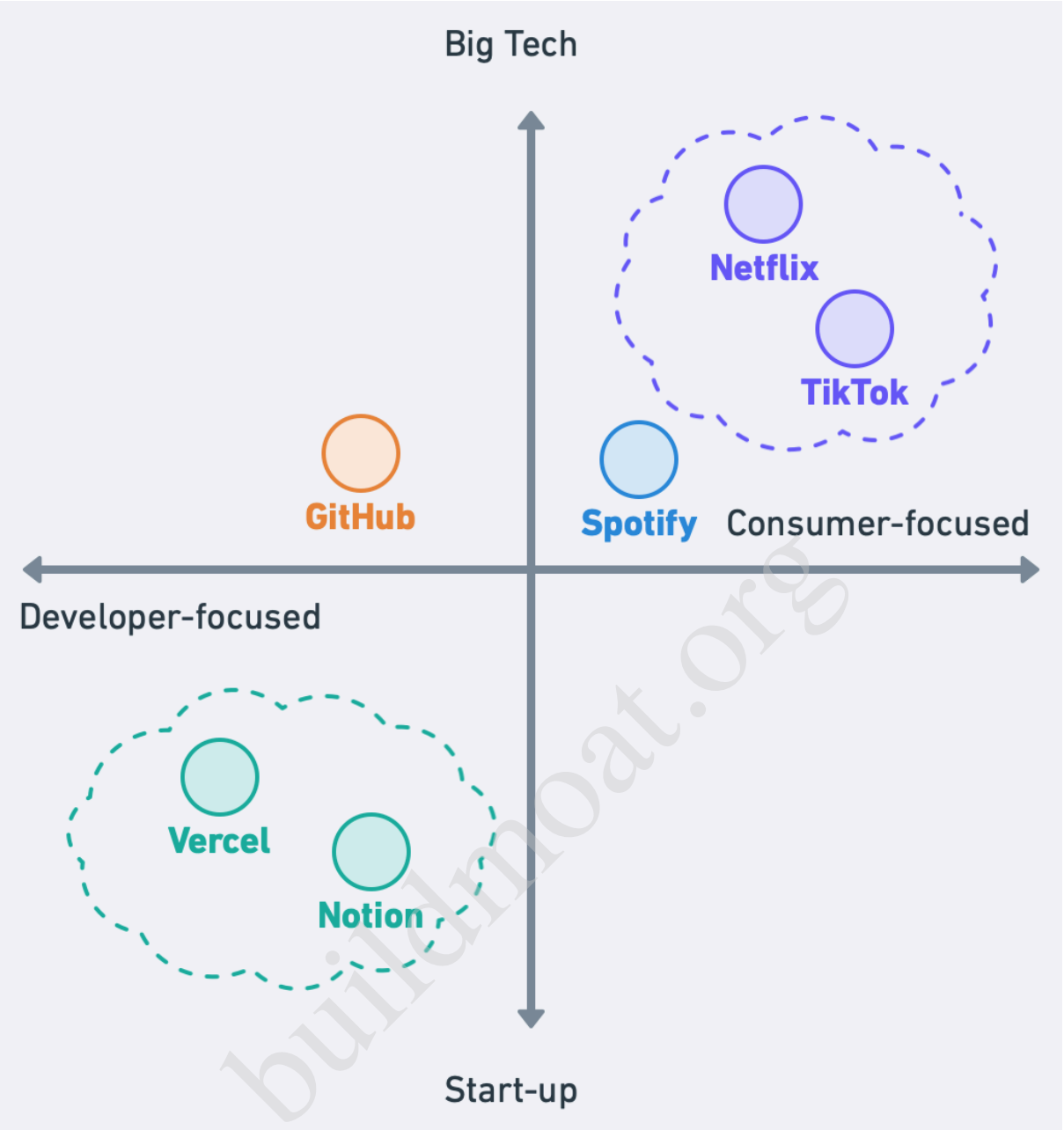
```
"The cat sat on the mat"
→ [0.12, -0.34, 0.78, ..., 0.45] (1536 dimensions)

"A feline rested on a rug"
→ [0.11, -0.32, 0.79, ..., 0.44] (非常接近)

"The stock market crashed"
→ [-0.89, 0.12, -0.45, ..., 0.23] (差很遠)
```

一般 embedding 維度 (dimension) 常見 128～1536。OpenAI `text-embedding-3-large` 是 3072 維。每一維通常無法被人類直觀理解，但重點不在單一維度，而是「幾何關係」能反映語意關係。

buildmeat.org



至於「相似」是什麼？其實取決於 embedding 模型。

常見做法：

文字 → OpenAI embedding API、Sentence Transformers、BERT
圖片 → CLIP、ResNet

這些模型通常學到一種「語意上的相似」。

但在推薦系統裡，相似也可能是「常一起被購買」。例如尿布和奶瓶，在語意上不太像，但對新手爸媽來說非常相關。這時候可以訓練專門的模型，讓 embedding 反映「共購行為」。

相似度怎麼算？

有向量之後，要定義相似度。常見幾種方式：

1. Euclidean Distance（L2 距離）

   就是高中學的畢氏定理延伸到高維。距離越小代表越相似。會考慮方向與大小。

2. Cosine Similarity：看兩個向量的夾角。大部分 embedding 都會 normalize，因此 cosine similarity 很常見。

   a. 同方向 → 1

   b. 垂直 → 0

   c. 反方向 → -1

3. Dot Product

   類似 cosine similarity，但不做 normalization。計算稍快。

4. Hamming Distance

   用於 binary 向量。計算 XOR 後統計 bit 差異數量。非常快。

在 infra 型系統設計面試中，只要說「我們會用 cosine similarity 或適合該模型的 metric」通常就夠了。

Nearest Neighbors 問題

K-Nearest Neighbors（KNN）問題：

給一個 query vector，找出最相似的 K 個向量。

最簡單做法：

對資料庫中每個向量都計算相似度，排序，取前 K 個。這樣的做法時間複雜度 O(n)。

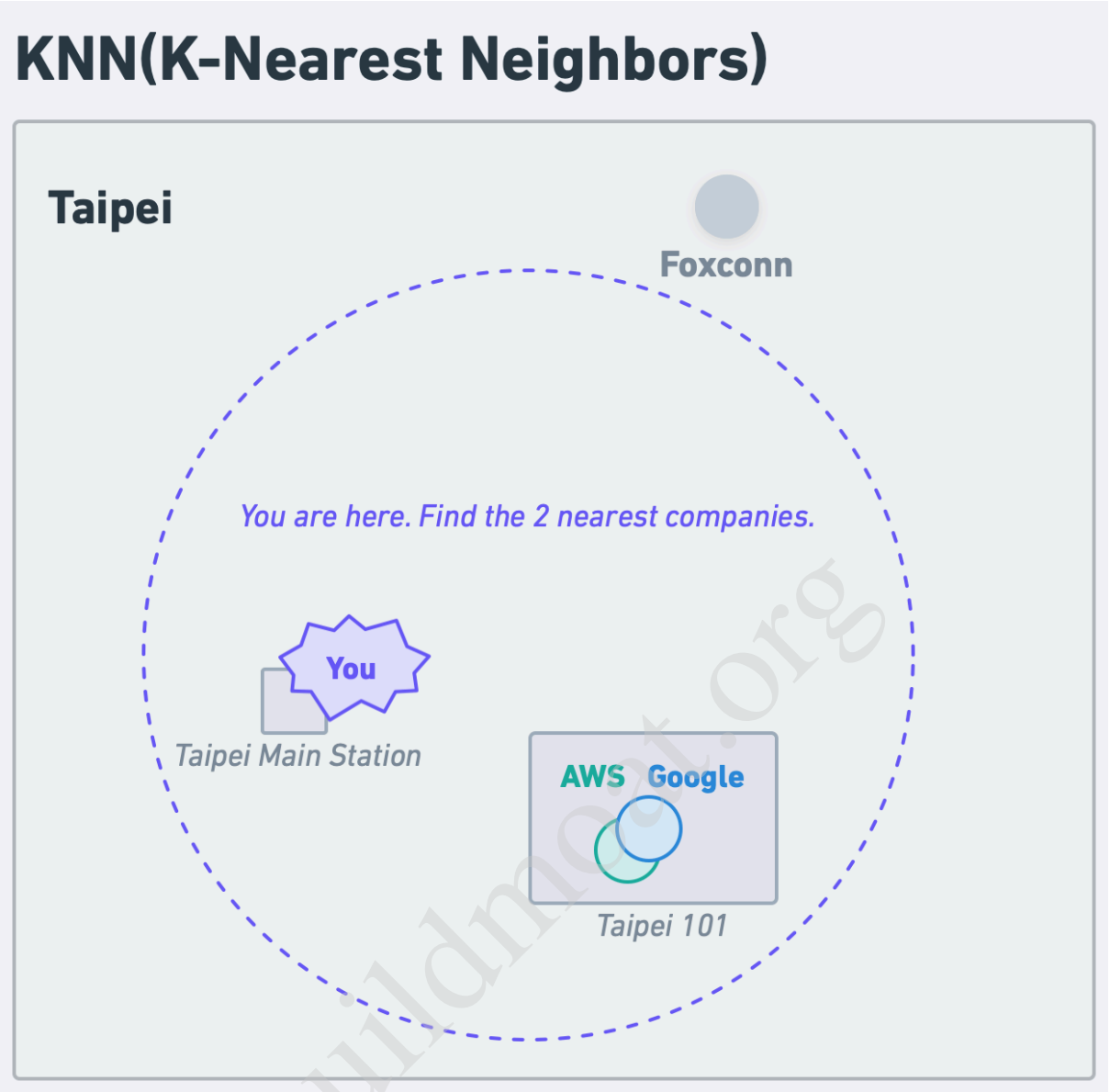
假設：

1M vectors，每個 1536 維

每次 query 要做約 60 億次浮點運算，非常昂貴。

雖然可以用 SIMD 或 GPU 加速，但對大規模系統仍然不夠。

buildmoat.org



Approximate Nearest Neighbor（ANN）

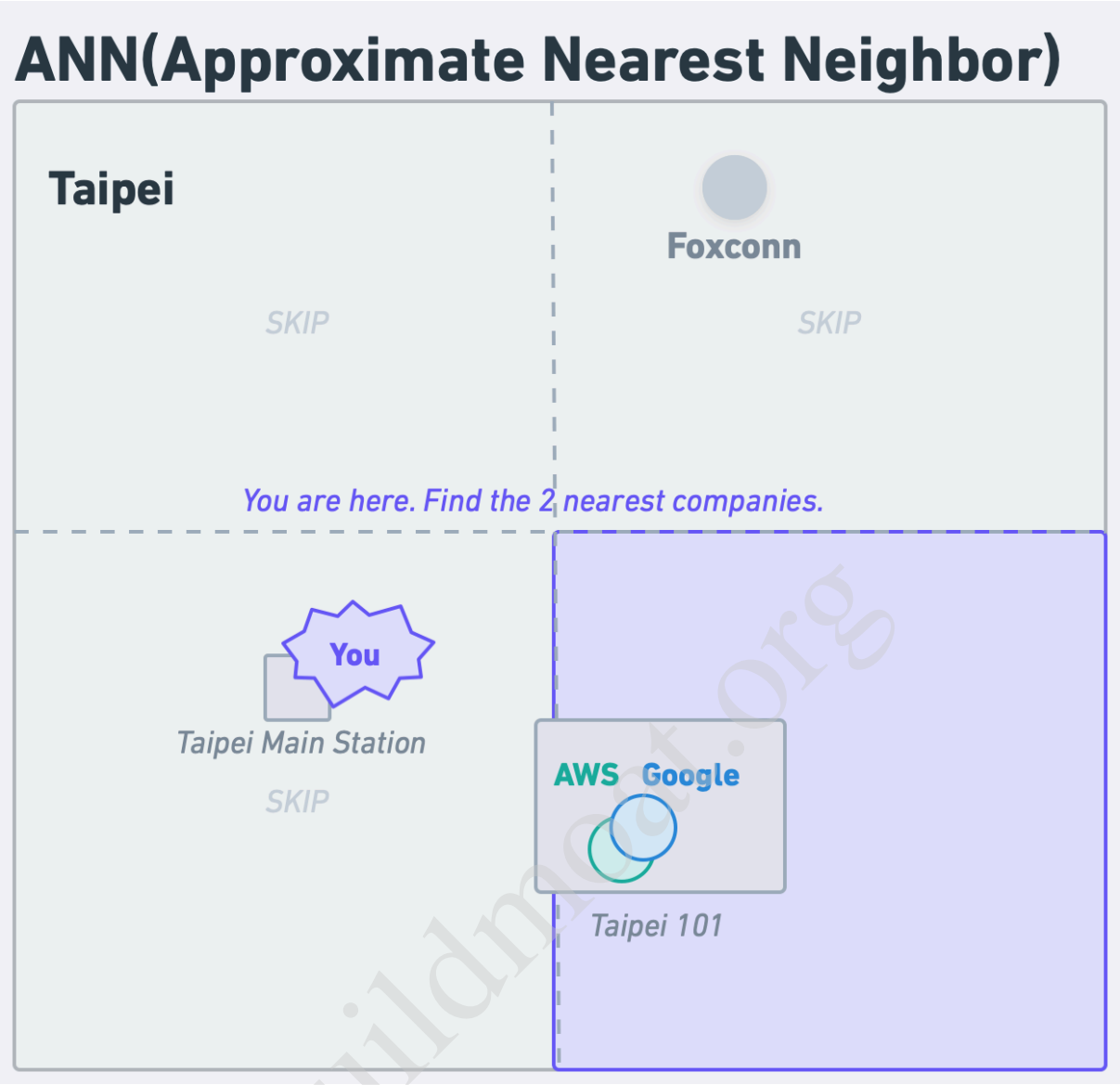
大多數應用可以容忍「不是完全準確」。這就進入 ANN（Approximate Nearest Neighbor）：

我們不一定找到真正的 top-K，但找到 95% 以上就夠。

核心 tradeoff：

- Recall（準確率）
- Latency（延遲）
- Memory（索引佔用記憶體）

向量資料庫本質上就是讓你在這三者間做平滑調整。



buildmeet.org

索引策略

HNSW（Hierarchical Navigable Small World）

這是目前最主流演算法。可以把它想成「高維度 skip list」。

概念：每個向量是 graph 上的一個節點，節點會和「相似向量」連邊，形成多層 graph

底層：所有向量

上層：隨機抽樣節點，當作「高速通道」

搜尋方式：

1. 從最上層開始

2. greedy 往最接近 query 的節點走
3. 走到不能更近為止
4. 往下一層
5. 最後在底層做較完整搜尋

優點：

- O(log n) 搜尋
- 95%+ recall
- 低延遲

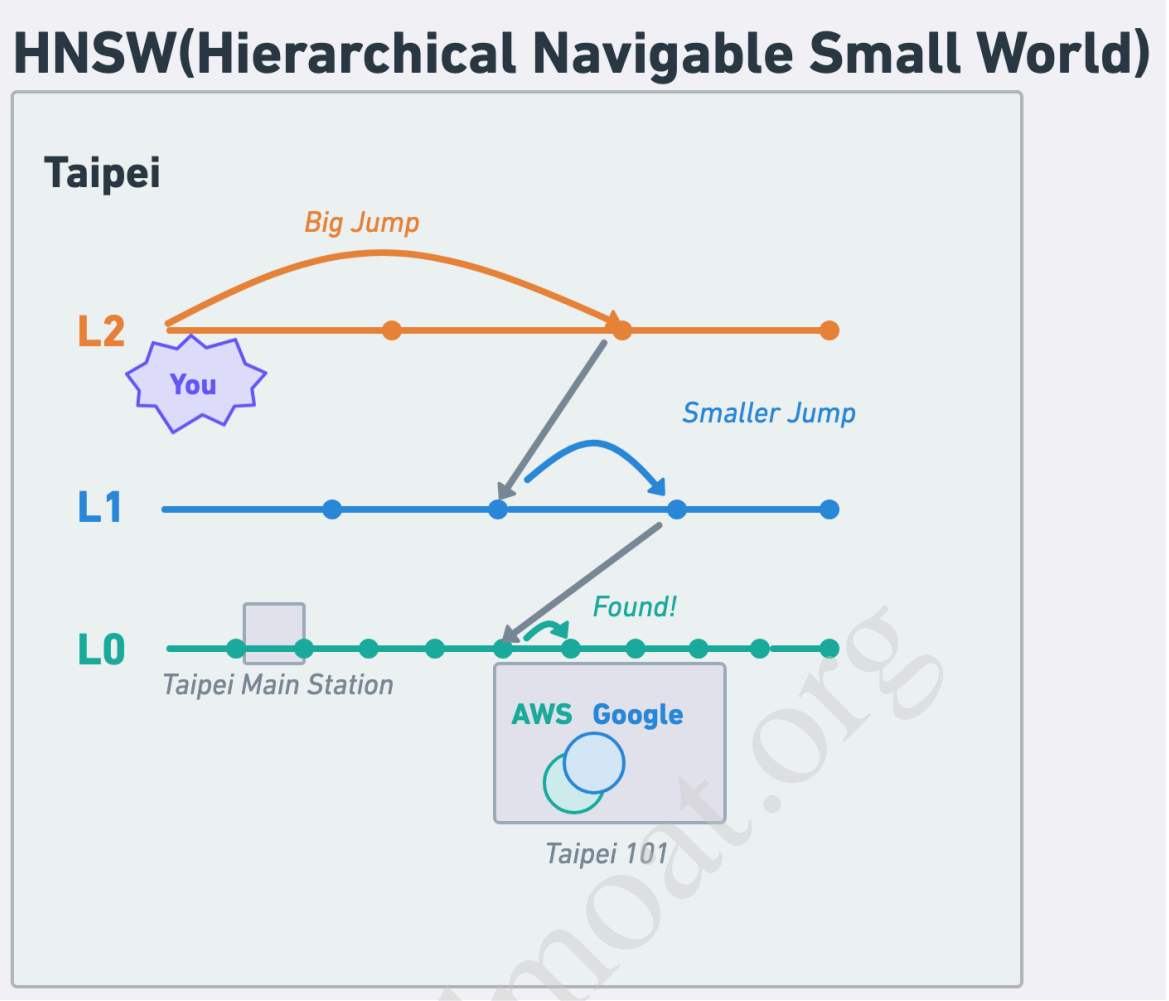
缺點：

- 記憶體大（約原始向量 2 倍）
- 建索引慢
- 插入昂貴

面試時可以說：

「使用 HNSW，多層 graph，從稀疏層往密集層 greedy search。」

buildmoat.org



IVF（Inverted File Index）

思路：先用 k-means 把向量分群，每個群有各自的 centroid，查詢時先找最近 centroid，並只在那些 cluster 中搜尋。

參數 `nprobe`：決定要搜尋幾個 cluster

高 `nprobe` → 高 recall 但慢

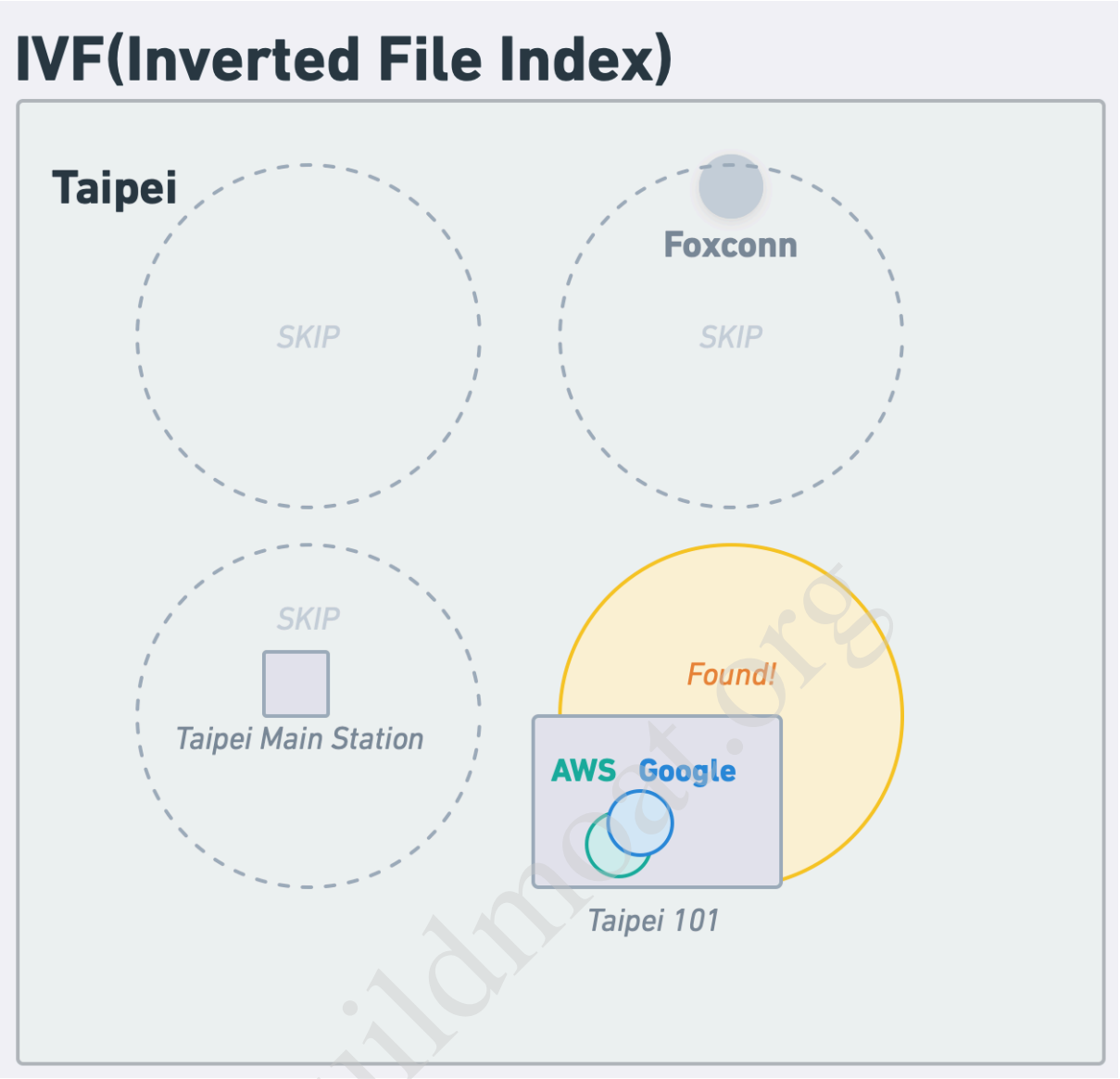
優點：

- 建索引快
- 插入簡單
- 記憶體小

缺點：

- recall 通常比 HNSW 低

buildmoat.org



LSH（Locality Sensitive Hashing）

設計 hash function，使相似向量容易落在同一 bucket。

做法：隨機畫 hyperplane，對於一個 hyberplane，向量在某側 → 0，另一側 → 1

多個 hyperplane → 多 bit hash

查詢時：

算 query 的 hash，只比對相同 bucket

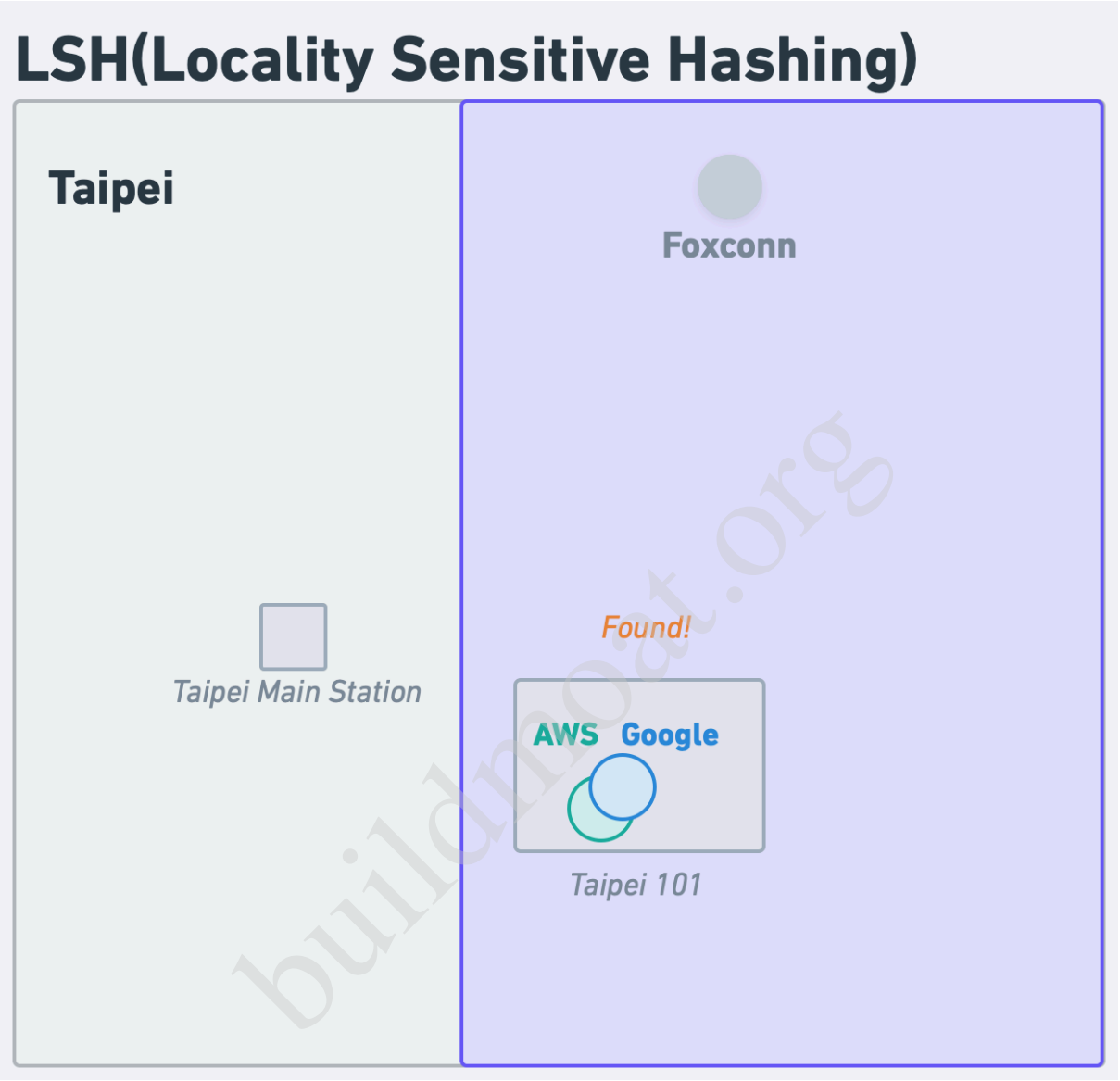
優點：

- 建索引快
- 適合 streaming
- 理論保證佳

buildmoat.org

缺點：

- 實務 recall 不如 HNSW



Filtering 與 Hybrid Search

實務需求很少是：「找最相似的 10 個」

通常會加條件：找最相似的 10 個，而且要 in stock，而且價格 < $100，而且今年發布。

兩種策略：

1. Post-filter：先找 top-N，再過濾。問題：可能過濾後不足 K 個
2. Pre-filter：先過濾，再做向量搜尋。問題：索引可能無法有效套用

實務上常 benchmark 再決定。

各系統做法：

- PostgreSQL + pgvector：使用 query planner，可能直接 brute force。
- Elasticsearch：在 HNSW traversal 中整合 filter，支援 BM25 + vector hybrid search。
- Pinecone：metadata 當 first-class，索引內部同時處理 vector + metadata。

實務選擇建議

原則：先簡單。

不要一開始就用專門向量資料庫。

傳統資料庫 extension

- pgvector（PostgreSQL）
  - 支援 HNSW / IVF
  - 可 JOIN
  - ACID
- Elasticsearch kNN
  - 適合 hybrid search
- Redis Vector Search
  - 低延遲
- S3 Vector
  - AWS 方案

專門向量資料庫（> 100M vectors）

- Pinecone： Fully managed
- Weaviate： Open source
- Milvus： 大規模（billions）
- Qdrant： open source，過濾支援強
- Chroma：適合 prototype / RAG

buildmoat.org

在面試中使用向量資料庫

面試常見場景

向量資料庫會出現在不少系統設計題目中，而且幾乎都是和 AI / ML 有關。通常你不會被這種題目嚇到，因為你大概早就知道自己是在面試和 AI / ML 相關的團隊，或整家公司本身就是以這個領域為核心。

如果是這種情況，常見的題型大概會落在以下幾種，而這些題目幾乎都「必然」會用到向量資料庫：

語意搜尋（Semantic search）

例如：「設計一個文件搜尋系統」或「設計一個程式碼搜尋工具」。
使用者輸入自然語言查詢，你需要找出相關文件。這是最典型的 embedding + 向量搜尋應用。

推薦系統（Recommendations）

例如：「設計一個商品推薦系統」或「設計一個內容推薦動態牆」。
目標是找出與使用者曾經互動過的內容相似的項目。通常會和協同過濾（collaborative filtering）結合使用。

圖片 / 影片相似度搜尋（Image / video similarity）

例如：「設計一個以圖搜圖功能」或「設計一個相似影片推薦功能」。
模式相同：先將媒體內容轉成 embedding，再搜尋相似的向量。

RAG 系統（Retrieval-Augmented Generation）

例如：「設計一個知識庫問答系統」，或任何涉及 LLM 搭配自有資料的題目。
向量搜尋負責找出相關文件，LLM 負責綜合並生成最終答案。

去重（Deduplication）

例如：「設計一個近似重複內容檢測系統」。
可能是抄襲偵測、找相似客服工單、或辨識重複商品列表。
做法是將項目轉成 embedding，找出在某個相似度門檻內的項目。

異常偵測（Anomaly detection）

例如：「設計一個詐欺偵測系統」。
將交易轉成 embedding，找出與正常模式「不相似」的交易。

buildmoat.org

即使你沒有 ML 背景，閱讀這些系統的高層設計拆解（ML System Design breakdown）仍然很有幫助。這能幫助你建立直覺：向量資料庫通常會出現在系統的哪個位置，以及它解決什麼問題。

架構模式（Architecture Patterns）

模式一：向量資料庫作為獨立服務

這是最常見的模式。你的應用程式先產生或取得 embedding，將它送到向量服務，向量服務回傳相似項目的 ID，然後你再去主資料庫查詢完整資料。
這種做法的好處是關注點清楚分離（separation of concerns）。

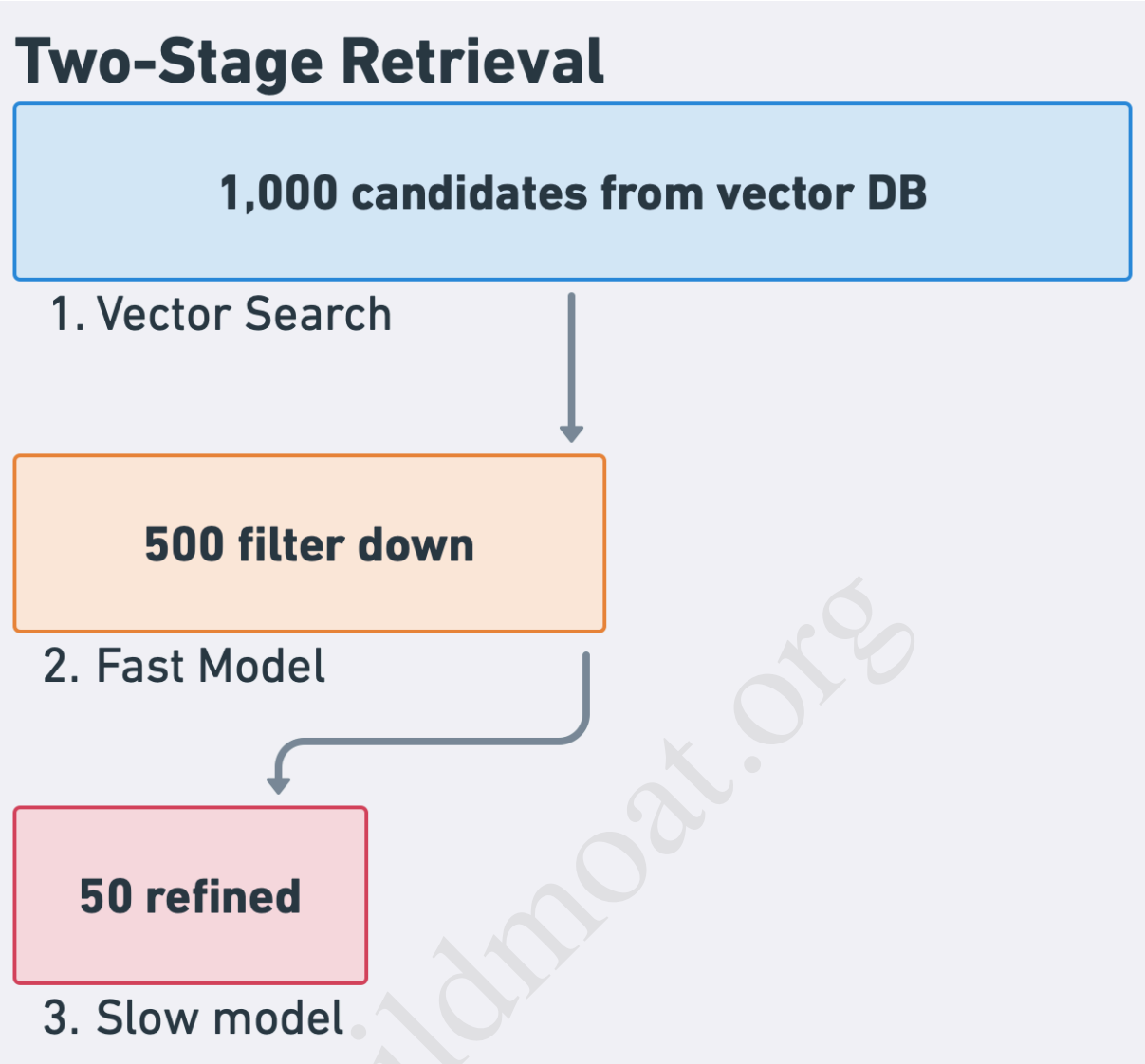
模式二：混合搜尋（Hybrid search）

查詢同時送到關鍵字索引（例如 Elasticsearch）和向量索引。最後用某種排序函數將結果合併。
這種模式適合搜尋應用，因為搜尋往往同時需要精確匹配與語意相似。

模式三：兩階段檢索（Two-stage retrieval）

第一階段：向量搜尋回傳一大批候選結果（例如前 1000 個）。
第二階段：使用更複雜（但較慢）的模型重新排序（rerank），挑出最終結果。
這在推薦系統中很常見，因為 reranker 可以利用 embedding 沒有捕捉到的額外特徵來做更細緻的排序。

buildneat.org



buildmoat.org

關鍵設計決策（Key Design Decisions to Discuss）

一致性需求（Consistency requirements）

向量搜尋的結果通常可以接受稍微「過時」一點。新加入的項目可能需要幾秒鐘，甚至幾分鐘後才能被搜尋到。你可以明確說出這點：「向量搜尋可以採用最終一致性（eventual consistency）；我們不需要在插入後立刻就能搜尋到該 embedding。」

更新策略（Update strategy）

Embedding 是如何進入系統的？是在項目建立時即時產生？還是透過每小時執行一次的批次任務（batch job）？這取決於你的延遲需求。如果對即時性要求不高，批次更新通常更實務。

過濾策略（Filtering strategy）

如果查詢包含條件過濾，你要怎麼處理？是 pre-filter、post-filter，還是混合方式？當過濾條件非常具選擇性時，這點特別重要，因為會直接影響效能與結果品質。

在偏基礎架構（infra-style）的系統設計面試中，比較少會深入討論以下主題：

Embedding 模型選擇（Embedding model selection）

是哪個模型產生 embedding？這會影響向量維度大小、品質與延遲。對文字任務，你可以簡單說：「我們會使用 sentence transformer 模型」或「使用 OpenAI 的 embedding API」。除非是 ML 系統設計面試，否則不需要深入細節。

索引類型（Index type）

一般來說回答「我們會使用 HNSW 以取得最佳查詢效能」通常是正確答案，除非你有特定限制（例如寫入極頻繁、資料規模極大、或記憶體受限）。

Embedding 更新（Embedding updates）

有時你會更換或升級 embedding 模型。這通常意味著要進行大規模重建索引，並可能需要精心安排切換流程。確保你的 API 中包含「是哪個模型產生此 embedding」的資訊，避免用錯模型的 embedding 來搜尋，是非常重要的設計細節。

Numbers to Know

以下是一些在面試中很實用的估算數字：

- Embedding 維度：常見範圍為 128–1536。OpenAI 常用 1536 維，許多開源模型使用 384 或 768 維。
- 每個向量的記憶體大小：float32 每維 4 bytes。1536 維向量約 6KB（比很多人直覺大）。
- 100 萬筆、1536 維向量：光原始向量約 6GB，再加上索引額外開銷（HNSW 大約會再乘以 2）。
- 查詢延遲（Query latency）：調校良好的系統可以做到 10ms 以下，常見為 1–5ms。
- Recall 目標：95% 以上通常可以接受。99% 以上也可達成，但會犧牲延遲或增加記憶體使用。
- 吞吐量（Throughput）：若索引常駐於記憶體，每個節點每秒數萬次查詢是合理的。

buildmost.org

常見陷阱與限制（Gotchas and Limitations）

向量資料庫不是交易型資料庫

不要把它當成系統的唯一資料來源（source of truth）。
它擅長做相似度搜尋，但不擅長傳統資料庫的其他功能。權威資料應該存放在其他主資料庫中，向量資料庫本質上是一種索引。

Embedding 漂移（Embedding drift）

如果你更換 embedding 模型，舊的 embedding 可能全部失效、不相容。
你必須重新產生所有 embedding（成本高），或在轉換期間維護多套索引。這是需要事先規劃的營運問題。

冷啟動（Cold start）

在個人化推薦中，如果你用使用者行為產生 embedding，新使用者沒有行為資料可用。
因此需要設計 fallback 策略。

維度與效能的權衡

高維 embedding 可以捕捉更多細節，但搜尋較慢、記憶體占用更高。
某些應用 128 維已經足夠，不要預設一定要使用模型提供的最大維度。

索引建構需要時間

在 1000 萬筆資料上建立 HNSW 索引，可能需要一小時甚至更久。
這會影響你部署變更或災難復原的速度。

向量搜尋不是精準匹配

如果你需要透過 ID 找到某篇文件，請使用一般資料庫。
向量搜尋找的是「相似」，不是「完全相同」。
有些場景兩種都需要。

總結（Summary）

向量資料庫讓我們能夠建立以「語意相似」為核心的新型應用，而不是只依賴精準匹配。

其核心技術是 Approximate Nearest Neighbor（ANN）搜尋，其中 HNSW 是目前生產環境中最常見的演算法。

buildmoat.org

實務建議是從簡單開始。如果你已經在使用 PostgreSQL，可以先嘗試 pgvector。只有當資料規模或需求超出現有擴充套件能力時，才考慮使用專門的向量資料庫。很多人低估了額外維運一套新系統的複雜度。

在面試中，向量資料庫在搜尋、推薦，以及各種 AI / ML 題目中越來越常見。你應該清楚知道：

- 它解決什麼問題（快速找到相似的東西）
- 它如何解決（使用 ANN 索引，例如 HNSW）
- 什麼時候適合用（語意相似，而不是精準匹配）

從簡單架構開始，只有在需求真的推動時再增加複雜度。

這個領域發展非常快。新的索引演算法、更緊密的資料庫整合方式，以及更好的工具持續出現。但不管最終哪個技術成為主流，embedding 表示資料、計算相似度，以及在 recall、latency、memory 之間做權衡這些基本原理，都會長期存在。

buildmoat.org